

Global Routing Optimization In Road Networks

Yehong Xu¹, Lei Li^{2,1,*}, Mengxuan Zhang³, Zizhuo Xu¹, Xiaofang Zhou^{1,2}

¹The Hong Kong University of Science and Technology, Hong Kong SAR, China

²The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China. ³Iowa State University, USA
yxudi@cse.ust.hk, thorli@ust.hk, mxzhang@iastate.edu, zxucj@connect.usk.hk, zxf@cse.ust.hk

Abstract—Route planning plays an increasingly important role in our society, and the routing results, which are the paths that vehicles actually travel in a road network influence the traffic condition naturally. However, the existing routing algorithms cannot consider the routing results and their influence simultaneously, so traffic congestion could be created when many vehicles are directed to follow similar routes. In this paper, we propose the *Global Routing Optimization* problem that aims to minimize traffic congestion by continuously evaluating traffic conditions for a set of routing tasks. It is non-trivial to achieve this global optimization goal, as routing and traffic condition evaluation are both time-consuming and interdependent. To break this dependency, we propose a global routing optimization paradigm that can evaluate the routing results’ influence on the traffic condition, and then plan the routes accordingly. To implement it, we first propose a *serial model* to optimize the next route, followed by a *batch model* to improve processing efficiency. After that, an *iterative model* is proposed to further optimize route qualities. Extensive experiments on large real-world networks with synthetic and real workloads validate the effectiveness and efficiency of our methods.

I. INTRODUCTION

Traffic congestion has been a ubiquitous problem in urban areas around the world which increases traveling time and further causes huge economic loss [1], contributes to air pollution [2], and deteriorates people’s life quality [3]. In this paper, we study the *Global Routing Optimization* problem which minimizes the total travel time of all the queries on a traffic-aware urban road network.

Motivation. Essentially, congestion happens on the road segments whose traffic flows are close to or greater than their capacity. In the past, traffic congestion was mainly attributed to the growing number of vehicles and the latent infrastructure construction. However, as an increasing number of users are relying on navigation apps, these apps become a congestion source unawaresly. This is because the routing algorithms behind them only optimize the route queries individually without considering their influence on the traffic flow, even though the future traffic condition is taken into consideration. Suppose we have a large set of routing queries that have similar sources, destinations, and departure times, the existing algorithms would assign similar routes to them. Then when the vehicles start traveling according to these routes, the traffic flow of these roads would increase dramatically and further trigger congestion, while the planned “optimal” travel time is never fulfilled [4]. In other words, it is the “local optimal” routes returned by the current algorithms that cause

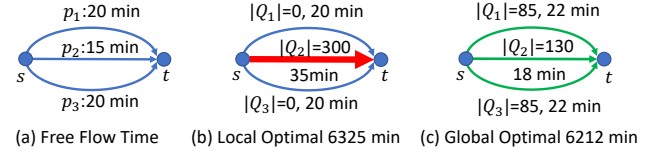


Fig. 1: Routing Optimization Example (Capacity is 100)

the congestion. Then a question arises naturally: could we make the routes “global optimal” such that the congestion could be eased?

For example in Fig. 1, (a) shows the free-flow travel time of three routes from s to t and we have 300 queries between them. The network is *traffic-aware* as the travel time is determined by the traffic flow through the *Bureau of Public Roads (BPR) function* [5]. The local optimal route is p_2 because it has the shortest expected travel time of 15 min, then all the queries are assigned to p_2 . However, because 300 is much larger than the route capacity of 100, the actual time would increase to 35 min, and the total travel time of the queries is 6325 min instead of the expected 4500 min, as shown in (b), with the longest travel time of 35 min. If we assign 85 queries to p_1 and p_3 each, then we could achieve a global optimal travel time of 6212 min (c), with the longest travel time of 22 min.

Existing Solutions. The global optimal results require two essential operations: *routing* that assigns queries to different paths, and *traffic evaluation* that returns the actual travel time. Unfortunately, the existing solutions cannot consider them at the same time, and we divide them into the following three categories: 1) *Passive Assignment* that provides a set of spatially dissimilar routes and assigns the queries to them randomly, which can be achieved by the *alternative / diversified routing* algorithms [6]–[13]. Although their results are better than the local optimal routing algorithms [14]–[21], they are still unaware of the results’ influence so they can hardly improve for better traffic condition; 2) *Traffic Assignment* [4], [22]–[26] that further assigns the queries to several pre-defined routes such that better traffic condition could be achieved by reaching the state of user-equilibrium [25], [26] where no traveler can find a cheaper route. However, [4] shows that user-equilibrium does not render the minimal total cost of travel. What’s worse, these methods are limited to the pre-found routes and cannot take advantage of the nearly unlimited other potential route options to further improve the traffic; 3) *Global Routing* [27]–[29] that considers the traffic influence during routing, but the state-of-the-art solution can only consider the influence of

* Lei Li is the corresponding author.

the next road because the influence on the remaining parts would be exponentially more complicated. In summary, the *Passive Assignment* lacks evaluation, the *Traffic Assignment* lacks routing, and *Global Routing* can only evaluate the next road during routing.

Challenges. The global optimal routing finds a travel plan with the minimum total travel time for a set Q of queries. To approach it, one may first let all queries take optimal paths based on current traffic conditions and identify the locations and time of congestion, then redirect some jammed ones to resolve the congestion. Some congestions will automatically vanish due to the change of some initial routes, nevertheless, the total travel time could increase because there may arise new congestions after detours. The cause of this phenomenon is that the traffic condition is determined by the routes, while the routes in turn are planned based on the traffic condition: they are intertwined such that the changes are propagated in tick-tock. Therefore, the challenge of global optimal routing is mainly featured in the following aspects: 1) We can hardly know the actual traffic condition before all routings reach their destinations because the actual traveling time along each road segment (edge weights) changes and is hard to capture during routing; 2) It is difficult to utilize the routing and optimization effectively simultaneously out of their coupling relationships; 3) It is hard to decide the rerouted routes to improve the traffic condition effectively and efficiently.

Idea. Unlike the existing solutions that utilize either routing or traffic evaluation or mix them together without a global view, in this work, we propose a new *global routing optimization paradigm*. Specifically, the routing and traffic conditions are detached and they run interchangeably such that routing could be more accurate and traffic conditions (edge weight) can be updated timely. Essentially, the paradigm works like the gradient descent algorithm in machine learning. To implement this paradigm, we first propose *serial model* that processes queries one by one under a certain order such that the previous routes' influence is considered in a small learning rate. Then, to improve the optimization efficiency, we propose the *batch model* by digging into the relations between the queries and identifying the ones that could be processed together in batches. Finally, to improve routing quality influenced by the query order, we further propose the *iterative model* which re-routes some queries according to the previous evaluation result.

Contributions. Our contributions are summarized below:

- 1) We provide a solid theoretical analysis in showing the complexity and obstacles of the global routing optimization problem and propose a new paradigm to solve it;
- 2) We propose three models to implement the paradigm: *serial model* that considers the previous queries' traffic condition for a given query order, the *batch model* that drastically improves optimization efficiency by identifying the batch processing opportunities, and the *iterative model* that approaches global optimal by selectively re-routing some queries;

- 3) We conduct extensive evaluations on the real-life road networks and queries to validate the effectiveness and efficiency of our approaches.

II. RELATED WORK

The related works consist of routing algorithms and congestion control algorithms because we study the congestion-control routing problem in this work. Specifically, the routing algorithms include single path finding algorithm (Section II-A1) and multiple path finding algorithm (Section II-A2).

A. Routing Algorithms

1) *Single Optimal Shortest Path Finding*: Most of the existing routing methods belong to this category, which returns one optimal path for a given OD (origin-destination pair), and they are the basis of modern navigation systems. Essentially, they differ in the edge weights: 1) *Shortest Path* assume the weights are static. They are the basis of all the remaining methods and can be categorized into *Index-Free Search* (Dijkstra's algorithm [14], A^* [30]), *Index-Based Search* (Arc-flag [31], [32], *ALT* [33], *CH* [34]), and *non-Search Hub Labeling* [16], [17], [35]–[39]. 2) *Dynamic Shortest Path* allows weight change to the latest traffic condition, which is obtained by monitoring the traffic, with *Batch Processing* [20], [21] for faster direct search and *Index Maintenance* [18], [19], [40]–[44] to utilize the index. 3) *Time-Dependent Fastest Path* [45]–[52] uses a function on each edge to tell different travel at different departure time such that the future traffic condition is considered, where the function is generated by statistic or prediction. 4) *Multi-Criteria Path* [53]–[59] allows each edge has multiple weights that represent different toll charge and fuel consumption. As we can see, although the edge weight types are evolving to approach the real-life, none of them consider the root cause (actual routes of the queries) of the traffic condition, so they cannot approach the global optimal and we treat them as baselines.

2) *Multiple Alternative Path Finding*: The alternative routing returns k paths of a given OD. The most fundamental algorithms are the *KSP* methods [60]–[65] that returns k paths in the strict length-increasing order. However, the returned paths are quite similar, so the *Diversified Top-k Paths* [7]–[9], [9]–[11] aims to apply similarity threshold among the results. As computing exact solutions are time-consuming, heuristic methods [6], [66]–[70] are proposed by using *via-node*, *plateau* and *penalties*. Nevertheless, their weights are still neither reflected in traffic conditions nor traffic assignment considers so they can only ease the traffic congestion passively but not actively.

B. Traffic Congestion Controlling

Most of the traffic congestion controlling methods change the edge weight by traffic influence but the routes are predefined and cannot change. The first kind is the *traffic assignment* that assigns each trip into several alternative paths at each instant of time based on certain behavior rules (user-equilibrium)

with game theories and operations research [24], [25], [71]–[77]. However, they are very time-consuming and can only work on toy networks with a few nodes. The second category treats the probability of the agent’s path choice as fractional flow and derives the probabilistic path decision vector for each traveler based on its neighbours’ path decisions [78], and the third uses a penalty to discourage users from entering the congestion roads [28], [29]. However, all the above methods are restricted by the predefined route set and cannot utilize the network flexibly. They are essentially the extension of previous alternative path methods and cannot approach the optimal. *NextOpt* [27] is the only work that integrates traffic influence into routing such that the weights are determined by the actual traffic flow, but it just optimizes the next road greedily so its effectiveness is limited.

III. PRELIMINARIES

In this section, we first describe the traffic-aware road network and its corresponding notions. Then we define the global routing optimization problem formally and analyze its hardness. Finally, we describe the baseline solutions.

A. Traffic-Aware Road Network

Different from the existing path finding problems where the roads have one single value (*static network* for shortest path [14]), or updatable values (*dynamic network* for dynamic shortest path [18]), or a function telling the travel time at different departure time (*time-dependent network* for fastest path [79]), or a timetable (*timetable network* for paths in public transportation [80], [81]), the travel time in our problem is determined by the actual traffic. In other words, we require the network to be *traffic-aware*, which is defined below:

Definition 1 (Traffic-Aware Road Network). A traffic-aware road network is represented as $G = (V, E, T)$ where V is the vertex set representing intersections, $E \subset V \times V$ is the edge set representing roads, and T is a traffic-aware function of each edge $(u, v) \in E$ that returns the actual travel time $T(u, v, t)$ depending on the actual traffic flow at different time t .

For each edge $(u, v) \in E$, we use $w(u, v)$ to denote its free-flow travel time from u to v , and $c(u, v)$ denote its capacity. Its travel time $T(u, v, t)$ is determined by $w(u, v)$, $c(u, v)$, and the traffic flow at t which is defined below:

Definition 2 (Traffic Flow). Given a road network G , the traffic flow $f(u, v, t)$ of an edge $(u, v) \in E$ at time t is the number of vehicles on (u, v) at t .

If there are other vehicles apart from the queried ones on the roads, we can add them together to obtain the traffic flow:

$$f(u, v, t) = f_q(u, v, t) + f_o(u, v, t) \quad (1)$$

where $f_q(u, v, t)$ are the queried ones and $f_o(u, v, t)$ are the other ones. In this paper, we test different ratios of $f_q(u, v, t)$ and $f_o(u, v, t)$ to demonstrate the influence of the controllable queries in real life scenarios.

A path from u to v is $p = \langle (v_0, t_0), \dots, (v_k, t_k) \rangle$, where $v_0 = u$, $v_k = v$, $(v_i, v_{i+1}) \in E$, and t_i is the time at v_i , $\forall 0 \leq i < k$. The travel time $T(p, t_u)$ of a path p departing at t_u is defined as $T(p, t_u) = \sum_{i=0}^{k-1} T(v_i, v_{i+1}, t_i)$, where $t_i = t_{i-1} + T(v_{i-1}, v_i, t_{i-1})$ and $t_0 = t_u$. Let $\mathcal{P}(u, v, t_0)$ denote all simple paths from u to v departing at t_u , then the fastest path is the one with the minimum $T(p, t_u)$.

A travel $q = (o, d, t)$ is a triplet of a source $o \in V$, a destination $d \in V$ and a departure time t . We use $Q = \{q_1, q_2, \dots, q_n\}$ to denote a set of such queries, and $P(Q) = \{p_1, \dots, p_n\}$ are their corresponding paths. Suppose all the travel time of these paths are accurate and the network only contains these queries Q , then the appearance of $P(Q)$ in G makes up the *traffic flow* of each edge at different times.

B. Problem Definition and Analysis

Because the traffic function $T(u, v, t)$ requires actual traffic flows of P to determine the exact values, we cannot know the actual travel time of the paths in P beforehand. Therefore, let us suppose we have an oracle process *TE* (*Traffic Evaluation*) for now to compute the travel time of all the paths in P that makes the travel time corresponds to the traffic flow for $T(u, v, t)$. Then for any fixed path set P , its paths $p_i \in P$ all have its determined travel time $T(p_i, t_i)$, and total travel time of these paths is $T(P) = \sum_{p_i \in P} T(p_i, t_i)$.

However, if we change the routes of $P' \subset P$, then not only P' travel time change, but some paths in $P \setminus P'$ might also be influenced. Consequently, the total time $T(P)$ also changes. Therefore, it would be beneficial to find the alternate paths in P such that $T(P)$ is minimal, which is defined as follows:

Definition 3 (Global Routing Optimization Problem (GRO)). Given a traffic-aware road network G and a query set Q , the GRO problem aims to find a set of paths $P(Q)$ such that $T(P(Q))$ is minimal.

Complexity. As discussed previously, any change of any path could lead to a different result, so in the worst case, we need to enumerate all the potential paths of any $q_i \in Q$ and call *TE* to compute $T(P)$. As proved in [82], [83], counting the number of paths between an OD is #P-Complete, which is at least as hard as NP-Complete, so the *GRO* problem is also at least NP-C. Moreover, [82] estimates an OD’s path number is $\Omega((|V| - 2)!d^{|V|-1})$, where d is the density of a random graph, then finding the optimal solution of *GRO* is $\Omega((|V| - 2)!d^{|V|-1})^{|Q|})$ times the complexity of *TE*. As the optimal solution has a prohibitively high complexity, we do not expect to find the exact optimal solution but resort to heuristic methods that are both effective and efficient in practice.

C. Baseline Solutions

In this section, we describe three baselines that could achieve lower $T(Q)$ than the local optimal algorithms.

B1: Search Space Limited Path Enumeration: The problem analysis aforementioned requires enumerating all the possible paths between an OD, which is neither practical nor necessary as most of them are much longer than the optimal. The

question is how we can reduce the path candidate enumeration space to find the optimal solutions faster. Here, we resort to *alternative routing* [6], [11], [13], [66], [67] that enumerates a small set of paths between an OD that are shorter than a distance threshold and are dissimilar enough from the optimal one. Suppose we enumerate k alternative paths for each query; then we have $k^{|Q|}$ combinations that are much smaller than the previous one. Furthermore, we can reduce the combination starting from the local optimal paths greedily and avoid some combinations if the corresponding paths are not influenced by others. The travel time is evaluated by *TE* and the minimum global travel time among the combinations is reported.

B2: Diversified Top-K Shortest Path with Traffic Assignment: The second baseline resorts to traffic assignment approaches [24], [26], [29], [78]. However, they require the candidate routes pre-defined such that the queries can be assigned to them. Therefore, we first cluster the queries based on the ODs. Then for each query cluster, we generate k candidate paths and form a sub-network from the paths of all query clusters. Finally, we call the traffic assignment algorithm to obtain the optimal result.

B3: Next-Step Optimal Routing: The last baseline only plans the route for each query once by evaluating the next road's traffic condition during path searching [27]. Specifically, all the queries are ordered in departure time and organized by a priority queue. Then each time, we retrieve the top one and expand to its neighbors with the best traffic condition (fast travel time) plus the shortest estimation travel time (pre-computed estimated travel time through the index). Although it has a better result as it considers the traffic of the next edge, the second part estimated travel time, is not the actual travel time, so it still cannot achieve the optimal.

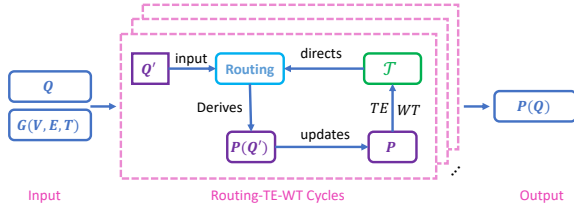


Fig. 2: Global Routing Optimization Paradigm

IV. GLOBAL ROUTING OPTIMIZATION PARADIGM

Our global routing optimization paradigm (Fig. 2) takes $G(V, E, T)$ and Q as inputs and outputs a route plan $P(Q)$. The paradigm has *Routing-TE(Traffic Evaluation)-WT(Weight Tuning) cycle* as a basic unit. It runs the *Routing-TE-WT* cycle repeatedly until terminate. Specifically, each cycle processes a subset of queries $Q' \subseteq Q$, which either have not been routed yet or its paths need improvement. Then *routing* first finds the optimal paths P' according to the current traffic condition $\mathcal{T}$ (initially with the free-flow traffic condition), *TE* evaluates the actual travel time of each path with the traffic-aware function, and *WT* updates $\mathcal{T}$ to the newest for the next cycle's routing. The updated weights by *TE-WT* embed the latest congestion information and guide the future routing to

avoid the current congestion (like the gradient update in the gradient descent). In this way, the routing can utilize the latest traffic condition and find the optimal solution globally (not just the next step road). In the following, we describe these three components in details.

A. Routing

In our paradigm, routing in the first *Routing-TE-WT* cycle uses the *Dijkstra's* algorithm with the free-flow travel time. Routing in all subsequent cycles uses the traffic condition $\mathcal{T}$ derived from the previous P that contains the newest traffic information so they can avoid congested regions and travel faster in practice. The details are shown in Algorithm 1. Different from the *Dijkstra's*, Algorithm 1 takes the departure time as the initial "distance". Besides, for an edge (u, v) , its travel time is returned by the time-dependent function $\bar{T}(u, v, t)$. Since we use the histogram-based function, the travel time retrieval operation takes constant time. Therefore, the traffic-aware routing of any query $q(o, d, t)$ has the same complexity as the *Dijkstra's*.

Algorithm 1: Fastest Path

Input: $G(V, E, T)$, $\mathcal{T}$, $q(o, d, t)$
Output: Fastest path p between o and d

```

1  $B[v] \leftarrow \infty, \pi[v] \leftarrow null, \forall v \in V(G)$   $\triangleright B$  stores time,  $\pi$  stores parent;
2  $B[o] \leftarrow t, L.Insert(o);$   $\triangleright L$  is a heap sorted by time
3 while  $L \neq \emptyset$  do
4    $u \leftarrow L.RemoveMin();$ 
5   if  $u = d$  then
6     break;
7   foreach  $v : (u, v) \in E$  do
8     if  $B[v] > B[u] + \bar{T}(u, v, B[u])$  then
9        $B[v] \leftarrow B[u] + \bar{T}(u, v, B[u]), \pi[v] \leftarrow u, H.Push(v);$ 
9  $p \leftarrow backTracking(\pi, d);$ 
10 return  $p;$ 
```

B. Traffic Evaluation TE

In this section, we introduce a traffic evaluation algorithm *TE* which computes the actual traffic flows and derives corresponding travel time with the help of traffic function $T(u, v, t)$. Essentially, $T(u, v, t)$ describes the relation between traffic flow and travel time. For ease of description, we use the below **Bureau of Public Roads (BPR)** function to establish this relation, which is widely used in the transportation area:

$$T(u, v, t) = w(u, v) \times (1 + \sigma \times (\frac{f(u, v, t)}{c(u, v)})^\beta) \quad (2)$$

where σ and β are two coefficients with recommended values of 0.15 and 4.

When a vehicle vh enters a road e at time t , it is the traffic flow of e at t that determines the actual time for vh to go through e ; while vehicles that enter e after t do not influence this travel time. Therefore, we only need to visit the vertices along all the paths in a time-increasing order to determine each path's next road travel time. The details are shown in Algorithm 2. Specifically, we use a heap L to organize all path labels according to their traversal time in increasing order. The paths all start from their first vertex at their departure time.

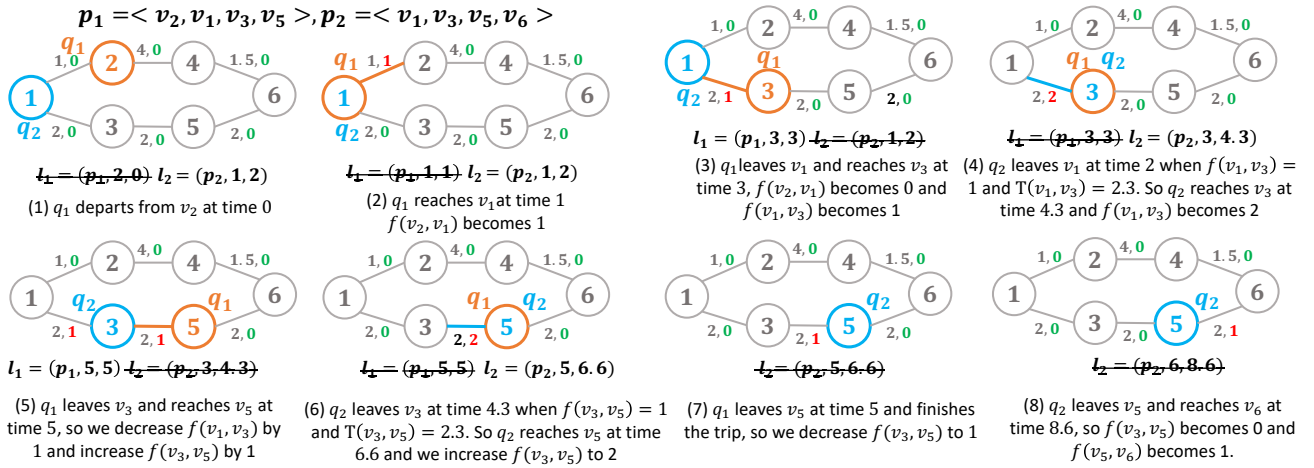


Fig. 3: An example of the traffic evaluation. Beside each edge is written two numbers where the black one denotes the free-flow travel time and the green one denotes the traffic flow of the edge. Capacities of all edges are set to 1.

Then each time we retrieve the earliest label l from L (line 3), remove it from the previous road and decrease its flow (line 7), and move it forward to the next road and increases its flow (line 9). If l has reached its path destination, then accumulate its travel time to $T(p)$. Otherwise, l is pushed back to L . When all the paths have finished their traversal, then we have obtained the total travel time.

Algorithm 2: Traffic Evaluation TE

Input: G, P
Output: $T(P)$

```

1  $L \leftarrow \{l(p) = (p, 0, p.t_0) \mid \forall p \in P\}; f(u, v) \leftarrow 0, \forall (u, v) \in E;$ 
   $T(P) \leftarrow 0;$ 
2 while  $L \neq \emptyset$  do
3    $l(p) = (p, i, t) \leftarrow L.RemoveMin();$   $\triangleright L$  is a heap ordered by  $t$ 
4   if  $i = |p|$  then
5      $T(P) \leftarrow T(P) + t - p.t_0$ ; Continue;  $\triangleright p$  finished traversal
6   if  $i > 0$  then
7      $f(v_{i-1}, v_i) \leftarrow f(v_{i-1}, v_i) - 1$ ;  $\triangleright$  Reduce previous road's flow
8      $t \leftarrow t + T(v_i, v_{i+1}, t); l(q) \leftarrow \{p, i + 1, t\}; L.Insert(l(q));$ 
9      $f(v_i, v_{i+1}) \leftarrow f(v_i, v_{i+1}) + 1$ ;  $\triangleright$  Increase next road's flow
10 return  $T(P);$ 

```

Example 1. As shown in Fig. 3, given a road network G and two travel queries $q_1 = (v_2, v_5, 0)$, $q_2 = (v_1, v_6, 0)$, suppose q_1 (resp. q_2) travels along the path between its OD with the minimum free-flow travel time: $p_1 = \langle v_2, v_1, v_3, v_5 \rangle$ ($p_2 = \langle v_1, v_3, v_5, v_6 \rangle$). L is initialized with $\{l_1 = (p_1, v_2, 0), l_2 = (p_2, v_1, 2)\}$. Because l_1 has an earlier time, it is removed from L and q_1 leaves v_2 (Fig. 3(1)). As the edge (v_2, v_1) is empty when q_1 leaves, q_1 reaches v_1 at time 1, its label becomes $l_1 = (p_1, v_1, 1)$ and is inserted into L . Besides, q_2 makes flow $f(v_2, v_1) = 1$ (Fig. 3(2)). In the following stages, q_1 and q_2 moves forward in turn, and L and edge flows are updated based on each movement (Fig. 3(2)-(6)). q_1 reaches its destination at time 5 (Fig. 3(7)), so $T(p_1, 0) = 5$. q_2 reaches its destination at time 8.6, so $T(p_2, 2) = 6.6$ (Fig. 3(8)). Therefore, we have $T(\{p_1, p_2\}) = 11.6$.

Theorem 1 (Traffic Evaluation Correctness). Algorithm 2

can compute the traffic time correctly.

Proof. We can prove it by induction.

(Basis) When the first path p with the earliest departure time starts, none of the other paths has started so the travel time of its first edge is based on the free-flow time and is correct. Besides, it has increased these roads' traffic flow by one.

(Induction) Suppose p is popped out of L at time t . All the path traversals before t have been processed, so all the road's flows are correct at time t . Then the next road's travel time of p is correct, and p has also decreased the previous road's flow and increased the next's flow. Therefore, we can guarantee the traffic flow is also correct after processing p . $\square$

Complexity. Because each path has at most $O(|V|)$ vertices, we have $O(|V||P|)$ times of *RemoveMin* operations. Because L has at most $|P|$ path labels, the time cost of *RemoveMin* is $O(\log |P|)$. Such that the total time complexity is $O(|V||P| \log |P|)$.

C. Edge-Weight Tuning WT

WT obtains the traffic condition $\mathcal{T}$ from current P and generate time-dependent functions $\bar{T}(\bullet)$ for subsequent routing. Intuitively, $\mathcal{T}$ can be obtained from the travel time of every path when they travel through the corresponding edges. From the edges' perspective, the # of the travel time information (in the form of (departure time, travel time)) equals the # of paths traversing through it. Although we can use this travel time information directly to generate time-dependent functions, it would be very zigzaggy and unnecessarily complicated for traffic-aware routing. Recall that the reason we adapt time-dependent functions is to expose congested roads to guide subsequent routing; we do not require them to exactly reflect the actual traffic condition. So we use histograms to collect and organize the travel time information for easier management and faster routing.

Specifically, we follow the real-life routing systems [84]–[86] to divide the time domain into 10-min slots. Thereby,

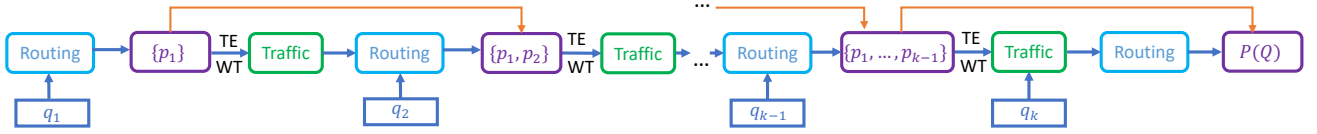


Fig. 4: Serial Optimization Strategies

each edge (u, v) is associated with a sequence τ of time slots: the non-empty ones take the average while the empty ones use the free flow time. Then (u, v) is associated with a piecewise time-dependent function $\bar{T}(u, v, t)$ which tells the approximate time required to travel from u to v at time t . The time-dependent functions $\bar{T}(\bullet)$ of all edges E collectively form a traffic condition $\mathcal{T}$ of underlying network G . Subsequent routing in the following cycle can run with $\mathcal{T}$.

Note that although we write the WT process after TE , it is only for clearer description while the time-dependent functions can be derived alongside TE as TE runs in the time-increasing order. Then when TE steps into a new time interval, we compute the average travel time of the previous interval for all the edges and start new collections for the coming interval.

Complexity. Each path contains at most $|V| - 1$ edges and each edge has τ slots. Considering that WT needs to update time-dependent functions of all traversed edges, the time complexity of WT is $O(\tau|V||P|)$.

D. Running Efficiency

Since queries input into the same *Routing-TE-WT* cycle faces the same input traffic condition, their routing can be performed in parallel. Therefore, the time complexity of routing Q' is the same as *Dijkstra's* algorithm for a single query, i.e. $O(|E| + |V| \log |V|)$. Considering the time complexity of TE ($O(|V||P| \log |P|)$) and WT ($O(\tau|V||P|)$), the running efficiency of a *Routing-TE-WT* cycle is bottle-necked by TE & WT when $|P|$ is large.

E. Serial Model

To implement the global routing paradigm, two more issues need to address: (1) the termination condition; and (2) the input query set Q' of every *Routing-TE-WT* cycle. In this section, we introduce the *serial model* (Fig. 4), where given a query order, queries are fed into the paradigm one by one individually into *Routing-TE-WT* cycle. This model terminates when the last cycle finishes and outputs the route plan for all queries.

Complexity. Since the input of each *Routing-TE-WT* cycle is a single query; the *serial model* contains $|Q|$ cycles. In the i^{th} cycle, there exist i paths in the road network $G(V, E, T)$. Therefore, the time complexity of TE and WT of the i^{th} cycle is respectively $O(|V|i \log i)$ and $O(\tau|V|i)$. Whereas the time complexity of routing in every cycle is $O(|E| + |V| \log |V|)$. Summing up them over all cycles, the time complexity of routing, TE and WT of the *serial model* are $O(|Q||E| + |Q||V| \log |V|)$, $O(|V||Q|^2 \log |Q|)$ and $O(\tau|V||Q|^2)$, respectively.

Given any query computation order, the *serial model* finds its corresponding traffic-aware routes, and different orders

have different total travel time. The following theorem proves there exists an optimal order:

Theorem 2 (Optimal Serial Order). *There exists a query computation order such that the serial model can achieve the global optimal routing solution.*

Proof. Given any two neighboring queries q_1 and q_2 on any road (u_1, v_1) with q_1 entering at $t_1^{u_1}$ earlier than q_2 entering at $t_2^{u_1}$, we prove that q_1 also enters any edge (u_2, v_2) at $t_1^{u_2}$ earlier than q_2 at $t_2^{u_2}$ by contradiction. Suppose they enter (u_1, v_1) later than (u_2, v_2) (the reverse case can be proved similarly), and $t_1^{u_1} = t_1^{u_2} + T(u_2, u_1, t_1^{u_2}) < t_2^{u_1} = t_2^{u_2} + T(u_2, u_1, t_2^{u_2})$. If $t_2^{u_2} < t_1^{u_2}$, then for the time-dependent function $T(u_2, u_1, t)$ from u_2 to u_1 , we have a case of departing later but arrives earlier, which violates our FIFO assumption. Therefore, given an optimal routing schedule, we can identify a query computing order by the appearance of the queries on the edges. $\square$

However, such an optimal query order can only be obtained after the global optimal is achieved, and we cannot get it beforehand. Therefore, we resort to evaluating a query order efficiently and approach the optimal order by partially re-ordering the queries. In the following, the batch model in Section V improves the efficiency and the *iterative model* in Section VI works on the re-ordering.

V. BATCH MODEL

The *serial model* where each *Routing-TE-WT* cycle processes one query at a time can approach the local optimal with a small step size. The advantage is that the *serial model* never oversteps; however, it has a long optimization term, making it prohibitive to real-life applications. In this section, we propose the *batch model*, which enhances the optimization efficiency without deteriorating the final route quality. Briefly, given a set of queries Q , the *batch model* first adopts a preprocessing stage that partitions Q groups $Q_1, \dots, Q_m$ where queries in the same group Q_i ($1 \leq i \leq m$) should not have severe conflict in the same roads at the same time. Then the query groups are fed into the *batch model* sequentially as the input of each *Routing-TE-WT* cycle. The workflow is depicted in Fig. 5. We introduce the intuition and design details of the *batch model* in the following subsections.

A. Conflict Analysis

To enhance the efficiency, we can put a set of queries Q' into the same *Routing-TE-WT* cycle and route them in parallel. However, this may deteriorate the global routing quality as we ignore the conflicts between queries in Q' .

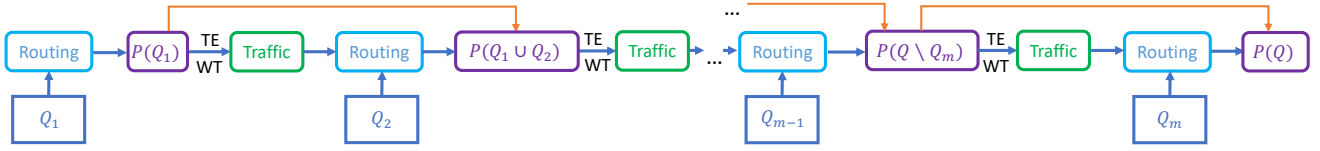


Fig. 5: Batch Optimization Strategies

For example, suppose two queries q_1 and q_2 are routed one after another by the *serial model* and generate two faraway two paths p_1 and p_2 . Then q_1 and q_2 are unlikely to have conflicts, so q_2 does not need to consider q_1 's result when routing, and q_1 and q_2 can run in parallel. In summary, by feeding non-conflicting queries into the same *Routing-TE-WT* cycles, we can significantly increase learning rate and shorten the routing term without spoiling the global routing quality.

B. Query Grouping

Our goal is to accelerate the learning rate and shorten the entire routing term while ensuring the routing quality. To achieve this, our *task* is to divide Q into as few # of groups as possible, with the compliance that queries within a same group do not involve severe conflicts.

Naturally there comes the question: how to evaluate the severity of conflicts of a given set of queries Q' ? Our solution is assuming the occurrence of a query $q(o, d, t) \in Q'$ over E distributes normally around the line segment $\bar{o, d}$. To evaluate conflicts within Q' , we compute the expected traffic flow $F_{Q'}(u, v)$ for each edge $(u, v) \in E$. Here, $F_{Q'}(u, v)$ is the sum of traffic flows contributed to (u, v) by all queries in Q' , defined as $F_{Q'}(u, v) = \sum_{q \in Q'} F_q(u, v)$, where $F_q(u, v)$ is a function of the shortest distance between q and (u, v) . We deem that Q' has a significant conflict issue if there exists an edge $(u, v) \in E$ such that $F_{Q'}(u, v) > \alpha \cdot c(u, v)$ where $\alpha > 0$ is a predefined parameter. The problem is that the time complexity of calculating $F_{Q'}(u, v), \forall (u, v) \in E$ is $O(E|Q'|)$, which is unacceptably high for road networks. Therefore, we do not implement it directly.

To improve the efficiency, we prune the edges far away from all queries in Q by decomposing G into a set of uniform grids (G_s) and calculating the expected traffic flow $F_{Q'}(g)$ of grids g that intercept or are nearby $\bar{o, d}$ of any $q(o, d, t) \in Q$. The capacity $g.c$ of a grid $g \in G_s$ is defined as the sum of capacities of all edges in g . We consider Q' to have a severe conflict issue if there exists a grid $g \in G_s$ such that $F_{Q'}(g) > \alpha \cdot g.c$.

Specifically, given a query set Q' , the grids G_s of the road network G and the parameter α , the discrete conflict evaluation algorithm *DisConEva* (Algorithm 3) (1) first finds the grids intercepted by the line segment $\bar{o, d}$ for each query $q(o, d, t) \in Q'$ (line 2); (2) then for each intercepted grid g , it retrieves 3 non-empty next to grids (line 8); (3) next, it updates $F_Q(g)$ of each intercepted/nearby grids g by accounting for q 's traffic flow contribution as shown in line 4 and 10-11 where $p(\bullet)$ is a normal distribution function $\mathcal{N}(0, \gamma^2)$. Algorithm 3 returns

true (line 6 and 13) if Q' has a severe conflict issue; otherwise, it returns false (line 15).

Algorithm 3: Discrete Conflict Evaluation *DisConEva*

Input: Q', α, G_s $\triangleright$ Grid size is γ^2
Output: *True* if Q' has a significant conflict issue; false otherwise

```

1 for  $q \in Q'$  do
2    $grids \leftarrow InterceptedGrids(q, G_s)$ ;
3   for  $g \in grids$  do
4      $F_{Q'}(g) \leftarrow F_{Q'}(g) + |g.e| \times p(\gamma)$   $\triangleright g.e$  denotes edges in  $g$ ;
5     if  $F_{Q'}(g) > \alpha \times g.c$  then
6       return True;
7    $i \leftarrow 1, g_l \leftarrow g.l, g_r \leftarrow g.r$   $\triangleright$  Get grids  $i$  block away from  $g$ ;
8   while  $i \leq 3$  do
9      $i \leftarrow i + 1$ ;
10     $F_{Q'}(g_l) \leftarrow F_{Q'}(g_l) + |g_l.e| \times p(\gamma \cdot i)$ ;
11     $F_{Q'}(g_r) \leftarrow F_{Q'}(g_r) + |g_r.e| \times p(\gamma \cdot i)$ ;
12    if  $F_{Q'}(g_l) > \alpha \times g_l.c$  or  $F_{Q'}(g_r) > \alpha \times g_r.c$  then
13      return True;
14     $g_l \leftarrow g_l.l, g_r \leftarrow g_r.r$ ;
15 return False;

```

We can build an inverted index offline that maps the edge $e \in E$ to the grid $g \in G_s$ where e resides. Thereby, operation *InterceptedGrids* takes constant time (line 2). We store grids G_s in a 2-dimensional array, which makes random access of grids (line 7) constant time. Besides, updating expected traffic flows (line 4 and 10-11) also takes constant time. Therefore, each for-loop runs in constant time. Consider that Algorithm 3 consists of $|Q'|$ such loops, its time complexity is $O(|Q'|)$.

With the help of *DisConEva*, our *task* can be formalized as the Query Selection Problem (QSP) introduced below.

Definition 4 (Query Selection Problem (QSP)). Given an existing set of travel queries Q , QSP aims to extract a subset $Q' \subseteq Q$ such that given the discrete road network G and the constant α : (1) *DisConEva* returns false for Q' ; (2) $|Q'|$ is maximized (to minimize the number m of query groups that Q need be divided into).

Condition (1) & (2) of QSP respectively consider the routing quality and efficiency issues of the *batch model*. QSP is essentially the Multidimensional Knapsack Problem (MKP) [87] (regarding every query $q \in Q$ as an item that weighs \$1, every grid $g \in G_s$ as a knapsack, and requiring that no knapsack can hold items more than its capacity). Consider that MKP is an NP-complete problem, addressing it is out of our scope. In this paper, we order the queries by their departure time and assemble as many consecutive queries into the same groups as possible with the limitation of condition (1).

Complexity. The input of each *Routing-TE-WT* cycle of the *batch model* is a unique group Q_i ($1 \leq i \leq m$). Let us assume

that every group has the same size $|Q|/m$, in the i^{th} cycle, there exists $i|Q|/m$ paths in the road network $G(V, E, T)$. Summing up over all m cycles, the time complexity of routing, TE and WT of the *serial model* is $O(m|E| + m|V| \log |V|)$, $O(m|V||Q| \log |Q|)$ and $O(\tau m|V||Q|)$, respectively. Given that $|Q| \gg m$, the overhead of query preprocessing adapted by the *batch model* can be overlooked when comparing its efficiency with the *serial model* efficiency.

VI. ITERATIVE MODEL

The *serial model* moves cautiously with small steps towards the local optimal solution. The *batch model* manages to improve the step size. However, both of their routing qualities are restricted by the given query order. In this section, we propose the *iterative model* to approach nearer to the global optimal. Briefly, we adopt the iterative model with the ‘re-routing’ mechanism. Unlike *serial model* and the *batch model* that route each query once, the iterative model repetitively re-routes some queries based on certain criteria. We thereby break the limitation imposed by a given query order. This provides us with chances to jump out of the local optimal routing solution.

The iterative model (Fig. 6) starts with a given set of paths $P(Q)$ for all travel queries Q and iteratively updates $P(Q)$ until termination. The model adapts a certain path selection strategy. Precisely, in the i^{th} iteration, the strategy selects an input query set Q'_i of current *Routin-TE-WT* cycle based on the traffic condition evaluated on the path set $P_i(Q)$ ($P_0(Q) = P(Q)$). After the cycle ends, we obtain the new path set $P_{i+1}(Q)$ by substituting. The algorithm terminates after a certain number of iterations. The path set P_{MIN} that has the minimal travel time throughout all the iterations is returned; as well as its total travel time.

Algorithm 4: Iterative Model

Input: $G, P(Q)$
Output: $P(Q), T(P(Q))$

```

1  $P_{MIN}, T_{MIN} \leftarrow \infty;$  ▷ Keep a record of optimal routing
2 repeat  $ITER\_NUM$  times
3    $P_r \leftarrow \text{PathSelection}(G, P(Q), \mathcal{F});$  ▷ Select the paths to re-route
4    $P' \leftarrow \emptyset;$ 
5   foreach  $p \in P_r$  do
6      $q \leftarrow q(p);$  ▷  $q$  corresponding to  $p$ 
7      $P' \leftarrow P' \cup \text{FastestPath}(G, q);$  ▷ Time-Dependent Re-Routing
8    $P(Q) \leftarrow P(Q) \setminus P_r \cup P';$ 
9    $T(P) \leftarrow \text{TE}(G, P(Q));$  ▷ Traffic Evaluation Algorithm 2
10   $\text{WT}(G, P(Q));$  ▷ Adjust temporal edge weights
11  if  $T_{MIN} > T(P)$  then
12     $P_{MIN} \leftarrow P, T_{MIN} \leftarrow T(P);$ 
13 return  $P_{MIN}, T_{MIN};$ 

```

Now, we consider the path selection problem. A road is congested because there are many paths passing through it. Therefore, to approach the global optimal, those paths $P_r \subset P(Q)$ that significantly prolong the total time should be re-routed. In the following, we discuss how to determine the P_r such that the total travel time of Q can be sufficiently reduced. Notably, we formalize the path selection problem as follows.

Definition 5 (Path Selection Problem). *Given a set Q of travel queries, an integer $k < |Q|$ and a solution path plan*

$P(Q)$, the path selection problem is to find a subset of k paths $P_r \subset P(Q)$ such that the remaining $T(P \setminus P_r)$ is minimized.

Similar to the *GRO* problem, the inter-dependency of the paths and the traffic condition makes the removal of any path determine a new traffic condition. To find the optimal solution, we have to test the influence of all the possible combinations of P_r , which still has a prohibitively high number of $\binom{|Q|}{k}$ candidate solutions to test. Therefore, we first propose a greedy method to approximate the optimal selection.

A. Greedy Selection Strategy

The details of the *PathSelection-Greedy* are shown in Algorithm 5. Specifically, during each iteration, we test the remaining paths P_m one by one and choose the one p^* that can reduce the total travel time the most as the next re-routing path. When k such paths are selected, we have obtained our P_r for re-routing.

Algorithm 5: PathSelection-Greedy

Input: $G, P(Q), k$
Output: P_r

```

1  $P_r \leftarrow \emptyset, P_m \leftarrow P(Q);$ 
2 while  $|P_r| < k$  do
3    $MIN \leftarrow \infty;$ 
4   foreach  $p \in P_m$  do
5      $T(P_m \setminus \{p\}) \leftarrow \text{TE}(G, P_m \setminus \{p\});$ 
6     if  $T(P_m \setminus \{p\}) < MIN$  then
7        $p^* \leftarrow p; MIN \leftarrow T(P_m \setminus \{p\});$ 
8    $P_r \leftarrow P_r \cup \{p^*\}, P_m \leftarrow P_m \setminus \{p^*\};$ 
9 return  $P_r$ 

```

Theorem 3 (Greedy Selection Approximate Bound). *Suppose the optimal selected path set is P_{opt} with the minimal remaining global cost $T(P \setminus P_{opt})$, then the result of Algorithm 5 is no bigger than $T(P \setminus P_{opt}) + \frac{1}{e}[T(P) - T(P \setminus P_{opt})]$.*

Proof. Firstly, minimizing $T(P \setminus P_r)$ is equivalent to maximizing $T(P) - T(P \setminus P_r)$. Then we can convert the path selection problem to maximizing the reducing power $T(P) - T(P \setminus P_r)$ problem. Next, we define the reducing power as a function $r(P_r) = T(P) - T(P \setminus P_r)$. Then we prove the following properties of $r(P_r)$:

Monotonicity: For any $P'_r \subseteq P_r$, we have

$$\begin{aligned} r(P_r) - r(P'_r) &= T(P) - T(P \setminus P_r) - (T(P) - T(P \setminus P'_r)) \\ &= T(P \setminus P'_r) - T(P \setminus P_r) \geq 0 \end{aligned}$$

because more traffic always has longer travel time.

Submodular: For any $P'' \subset P'$ and $p \in P \setminus P'$, we have

$$\begin{aligned} r(P'_r \cup \{p\}) - r(P'_r) &= (r(P_r \cup \{p\}) - r(P_r)) \\ &= T(P) - T(P \setminus (P'_r \cup \{p\})) - T(P) + T(P \setminus P'_r) \\ &= T(P) + T(P \setminus (P_r \cup \{p\})) - T(P) - T(P \setminus P_r) \\ &= [T(P \setminus P'_r) - T(P \setminus (P'_r \cup \{p\}))] \\ &= [T(P \setminus P_r) - T(P \setminus (P_r \cup \{p\}))] \end{aligned}$$

The above equation is non-negative because for the traffic function T , adding a new path to a larger set of paths would have higher overlapping with the existing paths such that the travel time grows faster. Since $P \setminus P_r \subseteq P \setminus P'_r$, the travel time gain of the smaller selection set P'_r is larger than P_r .

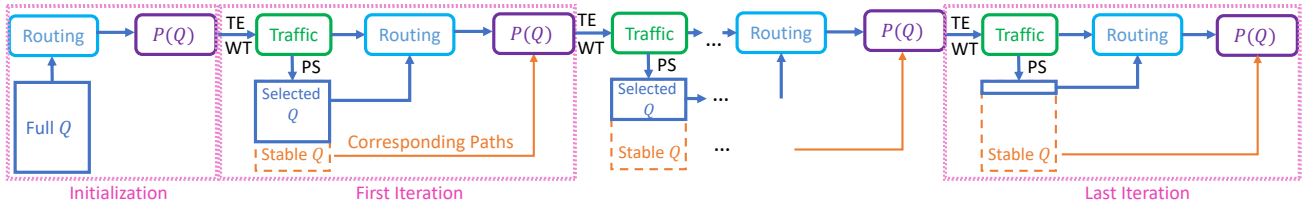


Fig. 6: Iterative Optimization Strategies

Because $r(P_r)$ satisfies the above properties, then it is a submodular function and has the following bound [88]:

$$\begin{aligned} r(P_r) &\geq (1 - \frac{1}{e})r(P_{opt}) \\ T(P) - T(P \setminus P_r) &\geq (1 - \frac{1}{e})(T(P) - T(P \setminus P_{opt})) \\ T(P \setminus P_r) &\leq \frac{1}{e}[T(P) - T(P \setminus P_{opt})] + T(P \setminus P_{opt}) \end{aligned}$$

Therefore, Theorem 3 is proved. $\square$

Complexity. The overall time complexity of Algorithm 5 is $\Pi_{i=0}^k (|Q| - 1)$ times $O(TE) = O(|V||Q|^{k+1} \log |Q|)$.

B. Greedy Selection Order

It is still very time-consuming to run Algorithm 5. Therefore, in the following, we resort to heuristic methods that have similar quality with higher efficiency. Ideally, the ordering reflects the extent of each path's contribution to the congestion of the entire path set P . By observing the paths selected by Algorithm 5, we propose a heuristic path selection paradigm in which existing paths are first sorted through the following orders, and then some top-ranked paths are selected as P_r .

1) *Random Ordering* $O_r(P)$. This method ranks given paths P randomly and add the paths ranked top- k to P_r .

2) *Latency-Based Ordering* $O_l(P)$. This method ranks given paths P in ascending order of *travel time latency*, which is the actual travel time minus the free-flow travel time. Paths ranked at top- k are added to P_r . The intuition is that paths that suffer significant time latency must pass severe congestion that should be replanned.

3) *Path-Based Ordering* $O_p(P)$. This method analyzes the paths P from the perspective of travel time latency and congestion. The ranking method is conducted in an iterative manner, where in each iteration, we (1) rank paths by $O_l(p)$; (2) remove the firstly ranked path from P and all congestion, and add p to P_r ; (3) label some congestion as 'addressed' if their related paths are less than a certain number; (4) update remaining paths' travel time latency by deducting their time spent on the 'addressed' congestion. To obtain a P_r of size k , we repeat the iteration k times.

4) *Congestion-Based Ordering* $O_c(P)$. The path-based ordering does not differentiate the importance of different congestion. However, some congestion is relatively more severe in the sense that it causes more traffic delays and results in other congestion. Therefore, we propose $O_c(P)$ by taking the congestion importance into consideration, which is also iterative where during each iteration: (1) Find the congestion that has the longest existing time; (2) Find the path p that

spends the most time on this congestion; (3) Remove p from P and all related congestion and, add it to P_r (4) Decrease existing time of related congestion. To obtain a P_r of size k , we repeat the iteration k times.

C. Three Models and Optimization Methods

In this section, we provide some insights into the proposed models and their relations from the perspective of the optimization methods. As proved in Theorem 2, our ultimate goal is following this order to achieve the global optimal. Like the *gradient descent* optimization method, which searches for the optimal solutions towards the largest gradient direction with a step length, our three models can also be viewed as an analog of gradient descent. Specifically, the *serial model* is like an optimization procedure with a random starting point, with *TE-WT* being the gradient computation and *Routing* being the step towards it. The *batch model* increases the step size from 1 to the number of queries within each batch for higher efficiency. The *iterative model's* re-routing is like re-selecting the starting point to jump out of the local optimal.

VII. EXPERIMENTS

In this section, we evaluated our *serial model* (*GRO-SER*), *batch model* (*GRO-BAT*), and *iterative model* (*GRO-ITE*).

A. Experiment Settings

Environment. We implemented all algorithms in C++ 17, compiled with GNU GCC 9.2.0 with full optimization, and on 10 servers each has 4 Xeon Gold 6248 2.6GHz CPUs (80 cores / 160 threads) and 1.5 TB memory running CentOS 7. **Datasets.** We conducted experiments on synthetic query sets and real-world query sets described as follows.

1) *Synthetic Query Sets.* We derived query sets varying in length and density as follows. We first randomly selected 15 seed ODs (u, v) that are 5 km, 10 km, and 40 km apart in the central area of the Beijing road network G_{BJ} where $|V(G_{BJ})| = 296,710$ and $|E(G_{BJ})| = 651,748$. Then for each (u, v) that is l km apart ($l = 5, 10, 40$), we found u 's (resp v 's) neighbor vertices $N(u)$ (resp. $N(v)$) in G_{BJ} and randomly selected 100 vertex pairs (o, d) such that $(o, d) \in N(u) \times N(v)$ and the shortest distance between o and d is about l km. We derived a query $q(o, d, 0)$ from every (o, d) with the departure time set to 0. We obtained 15 short, medium-length, and long query sets, each with a size of 100. We further derived 15 medium dense (resp. extreme dense) query sets by repeating each query of existing query sets once $\#rep = 1$ (resp. three times $\#rep = 3$). The synthetic query

(a) Avg. Travel Time (Total Travel Time / $|Q|$, unit: min)

(b) Avg. Optimization Time, unit: ms)

Query length γ km	Short			Medium			Long			Short			Medium			Long		
	0.25	1.00	4.00	0.25	1.00	4.00	0.25	1.00	4.00	0.25	1.00	4.00	0.25	1.00	4.00	0.25	1.00	4.00
Rep # = 0	6	6	10	10	12	23	33	33	45	9	3	2	15	4	2	75	17	6
Rep # = 1	12	15	64	18	39	103	37	44	146	9	2	1	15	4	1	77	19	5
Rep # = 3	43	36	1,856	65	110	1,357	112	149	1,185	9	3	1	16	4	1	80	20	5

TABLE 1: Effect of Grid Size γ (km) to *GRO-BAT* on Synthetic Query Sets

sets are used to evaluate each method’s performance under varying query length and sparsity.

2) *Real-world Query Sets*. We obtained taxi queries in G_{BJ} from DiDi and Manhattan road network G_{MH} ($|V(G_{MH})| = 11,368$, $|E(G_{MH})| = 36,792$) from New York City Council. In each road network, we randomly selected 50k taxi queries at midnight and repeated them once to mimic real-world traffic demands in low-traffic time in a day. Similarly, we randomly selected 33k (resp. 20k) queries in the early morning (resp. 7:00 am - 9:00 am) and repeated them 3 (resp. 5) times to mimic real-world traffic demands in normal time (resp. peak hours) in a day. Thereby, we obtained 6 query sets in total, in which 3 in Beijing and 3 in Manhattan. All query sets contain 100k queries with three different levels of sparsity.

Baselines. We chose the state-of-art alternative routing algorithm *AlterR* [7], diversified routing *DiverR* [8] and *NextOpt* [27] as baselines. Both *B1* and *B2* need a traffic assignment algorithm and we adopted the random allocation strategy because the existing traffic assignment algorithms cannot scale. We also compared our models to the congestion-unaware routing method *AllSPs*, which uses *Dijkstra*’s to derive the path with the minimum free-flow travel time for each $q \in Q$. The sum of their free-flow travel time (denoted by *Best*) is the unrealistic optimal result, and their traffic-aware travel time (*Local* evaluated by *TE*) is the bottom line routing quality.

Metric. Firstly, we show travel time for the quality. However, it should be noted that the travel time is highly influenced by the traffic evaluation function, whose quality is beyond this paper’s scope. Therefore, some large values indicate bad quality, but it does not necessarily mean the actual traveling time is that long. Secondly, we use the optimization time to show the efficiency because all the optimization methods have to consider all the queries together while the routings always run in parallel/distributed in real life to meet user’s need.

B. Effects of Parameters

1) *Batch Parameters*: *GRO-BAT* has two parameters: the grid size γ and the capacity threshold α , which influence the number m of groups Q would be divided into. Increasing γ or α may decrease the m , which could harm the quality of *GRO-BAT* but improve its efficiency. To validate this, we ran *GRO-BAT* with varying γ and fixed α on synthetic query sets.

As shown in Table 1b, *GRO-BAT* is the most efficient when $\gamma = 4.00$ km, and its efficiency decreases roughly linearly as γ increases. As the efficiency of *GRO-BAT* is proportional to m , we can infer that in this experiment, m is relatively small, medium, and large when $\gamma = 4.00$ km, 1.00

km, and 0.25 km, respectively. Meanwhile, the travel time is extremely high when $\gamma = 4.00$ km than $\gamma = 0.25$ km or 1.00 km, especially when $\#rep = 3$ (Table 1a). This shows the importance of separating conflict queries into different groups and effectiveness of Algorithm 3. Furthermore, the travel time under $\gamma = 1.00$ km is only slightly inferior to which under $\gamma = 0.25$ km, but the efficiency under $\gamma = 1.00$ km is much better (Table 1b). This proves that *GRO-BAT* can balance routing quality and efficiency when parameters are fine-tuned.

2) *Iterative Parameters*: *GRO-ITE* has three parameters: the number of iterations *ITER-NUM*; the number k of paths to re-route in each iteration; and the ordering of path selection. We ran *GRO-ITE* on the synthetic query sets with *ITER-NUM* set to 20, k set to different percentages of $|Q|$, and the path selection orders. The experiment reports the trends of total travel time under different settings over iterations on synthetic query sets. As shown in Fig.7, (1) *GRO-ITE* obtains faster improvements in routing quality over iterations with the increase of k from 0.5% to 10% of $|Q|$; however, if k is too large (30% of $|Q|$), its routing quality becomes unstable and deteriorates; (2) Random order leads to the most fluctuating quality and does not converge while others do; (3) Random order may hit better global routing solution, but when k is reasonably large (10% of $|Q|$), it is dominated by others; (4) The behaviors are similar among the latency-based, path-based and congestion-based orders; (5) *GRO-ITE* can improve routing quality on query sets with varying densities and lengths: as query length and density increase, the improvement becomes larger.

C. Regional Congestion Experiments

We conducted experiments on synthetic query sets to study the influence of different query lengths and sparsity. The results are presented in Table 2. *GRO-ITE* here is initialized with the output path set of *GRO-BAT*. The degree of conflicts among queries in a query set can be reflected by the difference between corresponding *Local* and *Best*: the larger the difference is, the more conflicts there are, and more prone is to congestion. From Table 2, we can find that almost all the query sets have conflicts, and the severity significantly increases as $\#rep$ and query length grows.

In terms of travel time (Table 2a), we find that all methods have low travel time on short and sparse ($\#rep = 1$) query sets. However, as conflicts become increasingly severe, performances of baseline methods deteriorate dramatically. For instance, long and dense ($\#rep = 3$) query sets reach up to 10 times longer than *Best*. In contrast, the travel time of our

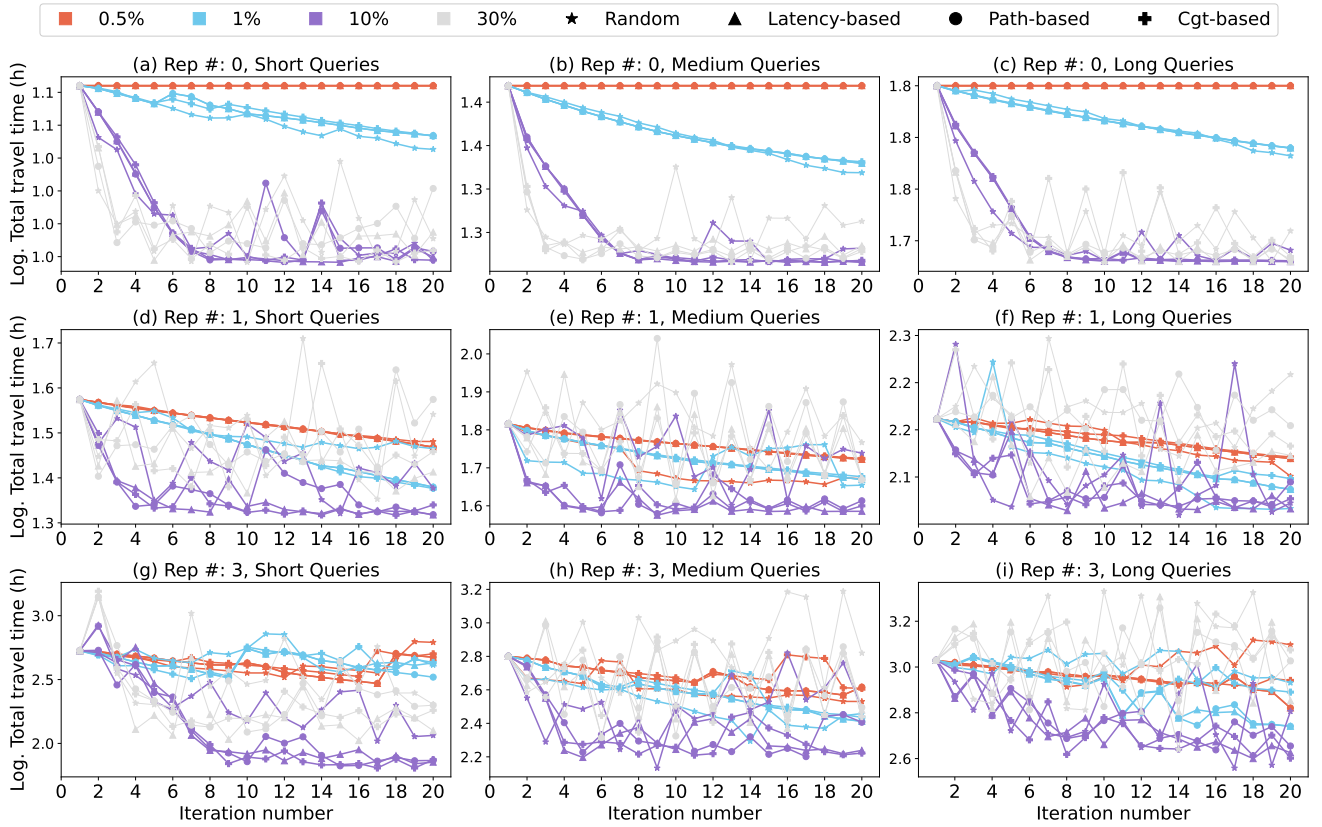


Fig. 7: Effect of k and Path Ordering on Synthetic Query Sets.

(a) Avg. Travel Time (Total Travel Time / $|Q|$, min)

Query length	Short			Medium			Long		
Rep #	0	1	3	0	1	3	0	1	3
Local	16	310	4,510	53	556	8,946	75	997	17,489
Best	6	6	6	9	9	9	30	30	30
GRO-SER	6	7	20	10	32	20	32	36	69
GRO-BAT	7	14	36	12	16	56	34	44	97
GRO-ITE	6	8	23	10	15	45	33	37	72
AlterR	8	13	159	16	73	111	74	149	337
DiverR	6	15	159	15	101	1,081	36	271	5,746
NextOpt	6	27	159	38	167	6,391	100	478	5,093

(b) Avg. Optimization Time (ms)

Short			Medium			Long		
0	1	3	0	1	3	0	1	3
-	-	-	-	-	-	-	-	-
24	24	26	42	44	48	101	94	96
8	8	8	9	9	9	10	10	10
8	9	10	10	10	13	11	12	13
6	3	2	6	3	2	50	25	13
17	14	7	15	8	3	28	8	4
0.4	0.3	0.3	1	0.8	0.7	14	14	13

TABLE 2: Performances of Varying Methods on Synthetic Query Sets.

methods is at most 3 times as long as *Best* even when $\#rep = 3$, showing that they scale well to long and dense queries.

In terms of the efficiency (Table 2b), *NextOpt* is much faster than other methods. However, its routing quality is much worse than *GROs*. *GRO-SER* is the most inefficient as it conducts *TE&WT* after routing each query, which makes it unscalable to large query sets. Therefore, we do not consider it in real-world experiment. *GRO-BAT* aims to improve the efficiency of *GRO-SER*. However, *GRO-BAT* pays substantial costs on routing quality. To remedy this, *GRO-ITE* adapts the re-routing mechanism based on *GRO-BAT*, which successfully improves the travel time and only slightly increases the running time. Besides, we observe that the running time of our methods increases slowly with the increase of the $\#$ queries, so our

methods can well scale to increasing query numbers.

D. Real-life Validations

We further use real-life query sets to validate the performances of *GRO-BAT* and *GRO-ITE* while *GRO-ITE* is dropped as it cannot scale. Regarding the routing quality (Table 3a), *GRO-ITE* manages to achieve the *Best*-level travel time under all traffic states in both Manhattan and Beijing. This implies the scalability of *GRO-ITE* in coping with query sets with different sparsity in road networks at different scales. *GRO-ITE* performs better than *GRO-BAT*, which highlights the importance of re-routing, and they are both much better than baselines. The baseline methods in general perform badly, showing their incapability in addressing the GRO problem. All

(a) Avg. Travel Time (Total Travel Time / $|Q|$, min)

(b) Total Optimization Time (sec)

Query length	Manhattan			Beijing			Manhattan			Beijing		
	Low	Normal	High	Low	Normal	High	Low	Normal	High	Low	Normal	High
<i>Local</i>	64,948	1,237,852	1,691,542	3,679	1,398,347	6,410,757	-	-	-	-	-	-
<i>Best</i>	33	34	33	16	16	16	-	-	-	-	-	-
<i>GRO-BAT</i>	39	41	42	23	27	29	7.2	6.7	6.5	60.3	58.7	65.2
<i>GRO-ITE</i>	35	34	35	16	16	16	7.3	8.5	7.3	53.0	50.2	42.6
<i>AlterR</i>	36	438	37	10,567	34,564	995,494	203.3	112.5	106.9	607.2	588.7	329.6
<i>DiverR</i>	35	3,134	11,060	938	707	9,414	162.5	109.5	93.7	488.8	460.2	250.8
<i>NextOpt</i>	35	67,356	64,504	76,794	52,202	2,142,912	2.0	2.0	2.0	533.2	518.9	545.6

TABLE 3: Performances of Varying Methods on Real-World Query Sets.

(a) Avg. Travel Time (Total Travel Time / $|Q|$, min)

(b) Total Optimization Time (sec)

Query length	Manhattan			Beijing			Manhattan			Beijing		
	0.2	0.4	0.8	0.2	0.4	0.8	0.2	0.4	0.8	0.2	0.4	0.8
<i>Local</i>	1,007,736	1,029,167	1,008,256	1,770,144	1,705,020	2,112,658	-	-	-	-	-	-
<i>Best</i>	33	33	33	16	16	16	-	-	-	-	-	-
<i>GRO-BAT</i>	53	47	39	81	34	18	2.8	4.5	3.9	11.7	20.7	32.3
<i>GRO-ITE</i>	52	45	35	48	32	17	4.2	5.4	6.5	18.9	25.1	40.8
<i>AlterR</i>	6,922	14,371	14,038	55,347	8,232	39,485	98.1	100.4	108.7	214.3	434.7	612.8
<i>DiverR</i>	20,312	3,932	5,720	3,516	3,691	2,563	84.3	84.1	93.3	226.2	313.8	459.2
<i>NextOpt</i>	94,477	533	75,877	1,971	1,852	1,058	2.7	3.3	4.3	330.5	430.6	558.1

TABLE 4: Scalability and Applicability of Varying Methods on Real-World Query Sets.

of them have large travel time in Beijing, showing that they scale badly to a large road network. When query numbers are different we show the average time, and when query numbers are the same we show the total time.

As for the efficiency (Table 3b), *NextOpt* and *AlterR* are the fastest, followed by *GRO-BAT* and *GRO-ITE*, and then *DiverR*. Both *GRO-BAT* and *GRO-ITE* are very efficient, using less than 10 sec (resp. 1 min) to process 100k queries in Manhattan (resp. Beijing). This shows the applicability of our methods to real-life scenarios.

E. Scalability and Applicability

This section studies the influence of non-queried routes on our methods, and the scalability of our methods to the # queries. Suppose there are 100k queries in Manhattan and Beijing during high traffic time, while only 20% (i.e. $f_q/f = 0.2$), 40 %, and 80% of them are known and planned by the route planning methods. We convert these unknown queries into larger free flow time and f_o , and only do the re-routing and evaluation on the known queries. As shown in Table 4a, traffic condition becomes better as the percentage of the known query increases because we have more opportunities to re-route and reduce congestion. In terms of efficiency, as shown in Table 4b, the total optimization time increases as the percentage increases. Nevertheless, it takes less than 10s to optimize 100k queries in Manhattan, and less than 1 min to optimize in Beijing, which is much faster than the traffic assignment baselines. Although *GROs* are slower than *GextOpt*, the quality is orders of magnitude better.

F. Summary

As validated by the results, our proposed methods can

achieve the best traffic condition compared with the baselines. Specifically, the serial model cannot scale when the query number is big, the batch model is fast but not sufficiently optimized, and the iteration model is a little bit slower with more optimization rounds but achieves higher-quality results.

VIII. CONCLUSION

In this paper, we identify the drawbacks of the existing routing algorithms in terms of the generated traffic congestion caused by similar routing results and further prolong the total travel time. Therefore, we aim to reduce the total time and propose the *global routing optimization* problem. The current solutions cannot solve this problem because the routing results would in turn affect the traffic condition and they are interdependent on each other. Therefore, we first propose our optimization paradigm that detaches the routing and traffic evaluation to provide optimization opportunities. After that, we propose a *serial model* to implement the paradigm sequentially, a *batch model* to improve the efficiency, and an *iterative model* to further approach the global optimal. Extensive experiment studies on both synthetic case studies and real-life scenarios validate the effectiveness and efficiency of our paradigm and models in easing traffic congestion.

ACKNOWLEDGMENT

The research work described in this paper was supported by Hong Kong Research Grants Council (grant# 16202722), Natural Science Foundation of China (grant# 62072125 and grant# 61902134) and was partially conducted in the JC STEM Lab of Data Science Foundations funded by The Hong Kong Jockey Club Charities Trust.

REFERENCES

- [1] G. Weisbrod, D. Vary, and G. Treyz, "Measuring economic costs of urban traffic congestion to business," *Transportation research record*, vol. 1839, no. 1, pp. 98–106, 2003.
- [2] F. A. Armah, D. O. Yawson, and A. A. Pappoe, "A systems dynamics approach to explore traffic congestion and air pollution link in the city of accra, ghana," *Sustainability*, vol. 2, no. 1, pp. 252–265, 2010.
- [3] K. Zhang and S. Batterman, "Air pollution and health risks due to vehicle traffic," *Science of the total Environment*, vol. 450, pp. 307–316, 2013.
- [4] T. Roughgarden and É. Tardos, "How bad is selfish routing?" *Journal of the ACM (JACM)*, vol. 49, no. 2, pp. 236–259, 2002.
- [5] U. S. B. of Public Roads, *Traffic assignment manual for application with a large, high speed computer*. US Department of Commerce, Bureau of Public Roads, Office of Planning, Urban . . . , 1964, vol. 2.
- [6] I. Abraham, D. Delling, A. V. Goldberg, and R. F. Werneck, "Alternative routes in road networks," *Journal of Experimental Algorithmics (JEA)*, vol. 18, pp. 1–1, 2013.
- [7] T. Chondrogiannis, P. Boursos, J. Gamper, U. Leser, and D. B. Blumenthal, "Finding k-shortest paths with limited overlap," *The VLDB Journal*, vol. 29, no. 5, pp. 1023–1047, 2020.
- [8] Z. Luo, L. Li, M. Zhang, Y. Xu, and X. Zhou, "Diversified top-k route planning in road network," *Proceedings of the VLDB Endowment*, 2022.
- [9] H. Liu, C. Jin, B. Yang, and A. Zhou, "Finding top-k shortest paths with diversity," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 3, pp. 488–502, 2017.
- [10] T. Chondrogiannis, P. Boursos, J. Gamper, and U. Leser, "Exact and approximate algorithms for finding k-shortest paths with limited overlap," in *20th International Conference on Extending Database Technology: EDBT 2017*, 2017, pp. 414–425.
- [11] T. Chondrogiannis, P. Boursos, J. Gamper, and U. Leser, "Alternative routing: k-shortest paths with limited overlap," in *Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2015, pp. 1–4.
- [12] C. Häcker, P. Boursos, T. Chondrogiannis, and E. Althaus, "Most diverse near-shortest paths," in *Proceedings of the 29th International Conference on Advances in Geographic Information Systems*, 2021, pp. 229–239.
- [13] L. Li, M. A. Cheema, H. Lu, M. E. Ali, and A. N. Toosi, "Comparing alternative route planning techniques: A comparative user study on melbourne, dhaka and copenhagen road networks," *IEEE Transactions on Knowledge and Data Engineering*, 2021.
- [14] E. W. Dijkstra *et al.*, "A note on two problems in connexion with graphs," *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [15] D. Delling, "Time-dependent sharc-routing," *Algorithmica*, vol. 60, no. 1, pp. 60–94, 2011.
- [16] T. Akiba, Y. Iwata, and Y. Yoshida, "Fast exact shortest-path distance queries on large networks by pruned landmark labeling," in *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, 2013, pp. 349–360.
- [17] D. Ouyang, L. Qin, L. Chang, X. Lin, Y. Zhang, and Q. Zhu, "When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 709–724.
- [18] M. Zhang, L. Li, and X. Zhou, "An experimental evaluation and guideline for path finding in weighted dynamic network," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2127–2140, 2021.
- [19] M. Zhang, L. Li, W. Hua, R. Mao, P. Chao, and X. Zhou, "Dynamic hub labeling for road networks," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 336–347.
- [20] M. Zhang, L. Li, W. Hua, and X. Zhou, "Stream processing of shortest path query in dynamic road networks," *IEEE Transactions on Knowledge and Data Engineering*, 2020.
- [21] L. Li, M. Zhang, W. Hua, and X. Zhou, "Fast query decomposition for batch shortest path processing in road networks," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 1189–1200.
- [22] S. Dafermos, "Traffic equilibrium and variational inequalities," *Transportation science*, vol. 14, no. 1, pp. 42–54, 1980.
- [23] M. Gugat, M. Herty, A. Klar, and G. Leugering, "Optimal control for traffic flow networks," *Journal of optimization theory and applications*, vol. 126, no. 3, pp. 589–616, 2005.
- [24] K. Kollias, A. Chandrashekarapuram, L. Fawcett, S. Gollapudi, and A. K. Sinop, "Weighted stackelberg algorithms for road traffic optimization," in *Proceedings of the 29th International Conference on Advances in Geographic Information Systems*, 2021, pp. 57–68.
- [25] J. G. Wardrop, "Road paper. some theoretical aspects of road traffic research," *Proceedings of the institution of civil engineers*, vol. 1, no. 3, pp. 325–362, 1952.
- [26] X. J. Ban, H. X. Liu, M. C. Ferris, and B. Ran, "A link-node complementarity model and solution algorithm for dynamic user equilibria with exact flow propagations," *Transportation Research Part B: Methodological*, vol. 42, no. 9, pp. 823–842, 2008.
- [27] K. Li, L. Chen, and S. Shang, "Towards alleviating traffic congestion: optimal route planning for massive-scale trips," in *Proceedings of the Twenty-Ninth International Conference on Artificial Intelligence*, 2021, pp. 3400–3406.
- [28] L. Chen, S. Shang, B. Yao, and J. Li, "Pay your trip for traffic congestion: Dynamic pricing in traffic-aware road networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, 2020, pp. 582–589.
- [29] L. Arp, D. van Vreumingen, D. Gawehns, and M. Baratchi, "Dynamic macro scale traffic flow optimisation using crowd-sourced urban movement data," in *2020 21st IEEE International Conference on Mobile Data Management (MDM)*. IEEE, 2020, pp. 168–177.
- [30] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [31] E. Köhler, R. H. Möhring, and H. Schilling, "Acceleration of shortest path and constrained shortest path computation," in *International Workshop on Experimental and Efficient Algorithms*. Springer, 2005, pp. 126–138.
- [32] M. Hilger, E. Köhler, R. H. Möhring, and H. Schilling, "Fast point-to-point shortest path computations with arc-flags," *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, vol. 74, pp. 41–72, 2009.
- [33] R. Bauer, D. Delling, P. Sanders, D. Schieferdecker, D. Schultes, and D. Wagner, "Combining hierarchical and goal-directed speed-up techniques for dijkstra's algorithm," *Journal of Experimental Algorithmics (JEA)*, vol. 15, pp. 2–1, 2010.
- [34] R. Geisberger, P. Sanders, D. Schultes, and D. Delling, "Contraction hierarchies: Faster and simpler hierarchical routing in road networks," in *International Workshop on Experimental and Efficient Algorithms*. Springer, 2008, pp. 319–333.
- [35] E. Cohen, E. Halperin, H. Kaplan, and U. Zwick, "Reachability and distance queries via 2-hop labels," *SIAM Journal on Computing*, vol. 32, no. 5, pp. 1338–1355, 2003.
- [36] Y. Li, M. L. Yiu, N. M. Kou *et al.*, "An experimental study on hub labeling based shortest path algorithms," *Proceedings of the VLDB Endowment*, vol. 11, no. 4, pp. 445–457, 2017.
- [37] W. Li, M. Qiao, L. Qin, Y. Zhang, L. Chang, and X. Lin, "Scaling distance labeling on small-world networks," in *Proceedings of the International Conference on Management of Data*, 2019, pp. 1060–1077.
- [38] A. W.-C. Fu, H. Wu, J. Cheng, and R. C.-W. Wong, "Is-label: an independent-set based labeling scheme for point-to-point distance querying," *Proceedings of the VLDB Endowment*, vol. 6, no. 6, pp. 457–468, 2013.
- [39] M. Jiang, A. W.-C. Fu, R. C.-W. Wong, and Y. Xu, "Hop doubling label indexing for point-to-point distance querying on scale-free networks," *Proceedings of the VLDB Endowment*, vol. 7, no. 12, 2014.
- [40] M. Zhang, L. Li, W. Hua, and X. Zhou, "Efficient 2-hop labeling maintenance in dynamic small-world networks," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 133–144.
- [41] D. Ouyang, L. Yuan, L. Qin, L. Chang, Y. Zhang, and X. Lin, "Efficient shortest path index maintenance on dynamic road networks with theoretical guarantees," *Proceedings of the VLDB Endowment*, vol. 13, no. 5, pp. 602–615, 2020.
- [42] T. Akiba, Y. Iwata, and Y. Yoshida, "Dynamic and historical shortest-path distance queries on large evolving networks by pruned landmark labeling," in *Proceedings of the 23rd international conference on World wide web*, 2014, pp. 237–248.
- [43] Y. Qin, Q. Z. Sheng, N. J. Falkner, L. Yao, and S. Parkinson, "Efficient computation of distance labeling for decremental updates in large dynamic graphs," *World Wide Web*, vol. 20, no. 5, pp. 915–937, 2017.
- [44] G. D'angelo, M. D'emidio, and D. Frigioni, "Fully dynamic 2-hop cover labeling," *Journal of Experimental Algorithmics (JEA)*, vol. 24, no. 1, pp. 1–36, 2019.

- [45] B. Ding, J. X. Yu, and L. Qin, "Finding time-dependent shortest paths over large graphs," in *Proceedings of the 11th international conference on Extending database technology: Advances in database technology*, 2008, pp. 205–216.
- [46] E. Kanoulas, Y. Du, T. Xia, and D. Zhang, "Finding fastest paths on a road network with speed patterns," in *IEEE International Conference on Data Engineering*. IEEE, 2006, pp. 10–10.
- [47] L. Li, W. Hua, X. Du, and X. Zhou, "Minimal on-road time route scheduling on time-dependent graphs," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1274–1285, 2017.
- [48] L. Li, K. Zheng, S. Wang, W. Hua, and X. Zhou, "Go slow to go fast: minimal on-road time route scheduling with parking facilities using historical trajectory," *The VLDB Journal*, vol. 27, no. 3, pp. 321–345, 2018.
- [49] L. Li, S. Wang, and X. Zhou, "Time-dependent hop labeling on road network," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 2019, pp. 902–913.
- [50] L. Li, S. Wang, and X. Zhou, "Fastest path query answering using time-dependent hop-labeling in road network," *IEEE Transactions on Knowledge and Data Engineering*, 2020.
- [51] Y. Wang, G. Li, and N. Tang, "Querying shortest paths on time dependent road networks," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1249–1261, 2019.
- [52] G. V. Batz, D. Dellling, P. Sanders, and C. Vetter, "Time-dependent contraction hierarchies," in *Proceedings of the Meeting on Algorithm Engineering & Experiments*. Society for Industrial and Applied Mathematics, 2009, pp. 97–105.
- [53] Z. Liu, L. Li, M. Zhang, W. Hua, P. Chao, and X. Zhou, "Efficient constrained shortest path query answering with forest hop labeling," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 1763–1774.
- [54] S. Wang, X. Xiao, Y. Yang, and W. Lin, "Effective indexing for approximate constrained shortest path queries on large road networks," *Proceedings of the VLDB Endowment*, vol. 10, no. 2, pp. 61–72, 2016.
- [55] S. Lu, B. He, Y. Li, and H. Fu, "Accelerating exact constrained shortest paths on gpus," 2020.
- [56] D. Dellling and D. Wagner, "Pareto paths with sharc," in *International Symposium on Experimental Algorithms*. Springer, 2009, pp. 125–136.
- [57] Y. Yang, H. Gao, J. X. Yu, and J. Li, "Finding the cost-optimal path with time constraint over time-dependent graphs," *Proceedings of the VLDB Endowment*, vol. 7, no. 9, pp. 673–684, 2014.
- [58] Z. Liu, L. Li, M. Zhang, W. Hua, and X. Zhou, "Multi-constraint shortest path using forest hop labeling," *The VLDB Journal*, pp. 1–27, 2022.
- [59] Z. Liu, L. Li, M. Zhang, W. Hua, and X. Zhou, "Fhl-cube: Multi-constraint shortest path querying with flexible combination of constraints," *Proceedings of the VLDB Endowment*, vol. 15, 2022.
- [60] J. Y. Yen, "Finding the k shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.
- [61] E. Q. Martins and M. M. Pascoal, "A new implementation of yen's ranking loopless paths algorithm," *Quarterly Journal of the Belgian, French and Italian Operations Research Societies*, vol. 1, no. 2, pp. 121–133, 2003.
- [62] J. Gao, H. Qiu, X. Jiang, T. Wang, and D. Yang, "Fast top-k simple shortest paths discovery in graphs," in *Proceedings of the 19th ACM international conference on Information and knowledge management (CIKM)*, 2010, pp. 509–518.
- [63] L. Chang, X. Lin, L. Qin, J. X. Yu, and J. Pei, "Efficiently computing top-k shortest path join," in *EDBT 2015-18th International Conference on Extending Database Technology, Proceedings*, 2015.
- [64] Z. Yu, X. Yu, N. Koudas, Y. Liu, Y. Li, Y. Chen, and D. Yang, "Distributed processing of k shortest path queries over dynamic road networks," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 665–679.
- [65] A. Sedeño-Noda, "Ranking one million simple paths in road networks," *Asia-Pacific Journal of Operational Research*, vol. 33, no. 05, p. 1650042, 2016.
- [66] D. Luxen and D. Schieferdecker, "Candidate sets for alternative routes in road networks," in *International Symposium on Experimental Algorithms*. Springer, 2012, pp. 260–270.
- [67] H. Döbler and B. Scheuermann, "On computation and application of k most locally-optimal paths in road networks," 2016.
- [68] P. Johnson, D. Joy, D. B. Clarke, and J. Jacobi, "Highway 3.1: An enhanced highway routing model: Program description, methodology, and revised users manual," Oak Ridge National Lab., TN (United States), Tech. Rep., 1993.
- [69] V. Akgün, E. Erkut, and R. Batta, "On finding dissimilar paths," *European Journal of Operational Research*, vol. 121, no. 2, pp. 232–246, 2000.
- [70] Y. Chen, M. G. Bell, and K. Bogenberger, "Reliable pretrip multipath planning and dynamic adaptation for a centralized road navigation system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 8, no. 1, pp. 14–20, 2007.
- [71] M. G. Bell and C. Cassir, "Risk-averse user equilibrium traffic assignment: an application of game theory," *Transportation Research Part B: Methodological*, vol. 36, no. 8, pp. 671–681, 2002.
- [72] T. L. Friesz, D. Bernstein, T. E. Smith, R. L. Tobin, and B.-W. Wie, "A variational inequality formulation of the dynamic network user equilibrium problem," *Operations research*, vol. 41, no. 1, pp. 179–191, 1993.
- [73] C. Gawron, "An iterative algorithm to determine the dynamic user equilibrium in a traffic simulation model," *International Journal of Modern Physics C*, vol. 9, no. 03, pp. 393–407, 1998.
- [74] B. Javani, A. Babazadeh, and A. Ceder, "Path-based capacity-restrained dynamic traffic assignment algorithm," *Transportmetrica B: Transport Dynamics*, vol. 7, no. 1, pp. 741–764, 2019.
- [75] P. Kachroo and N. Shlayan, "Dynamic traffic assignment: A survey of mathematical models and techniques," in *Advances in Dynamic Network Modeling in Complex Transportation Systems*. Springer, 2013, pp. 1–25.
- [76] W. Szeto and S. Wong, "Dynamic traffic assignment: model classifications and recent advances in travel choice principles," *Open Engineering*, vol. 2, no. 1, pp. 1–18, 2012.
- [77] O. Perederieieva, M. Ehrgott, A. Raith, and J. Y. Wang, "A framework for and empirical study of algorithms for traffic assignment," *Computers & Operations Research*, vol. 54, pp. 90–107, 2015.
- [78] S. Lim and D. Rus, "Stochastic distributed multi-agent planning and applications to traffic," in *2012 IEEE International Conference on Robotics and Automation*. IEEE, 2012, pp. 2873–2879.
- [79] L. Li, J. Kim, J. Xu, and X. Zhou, "Time-dependent route scheduling on road networks," *SIGSPATIAL Special*, vol. 10, no. 1, pp. 10–14, 2018.
- [80] H. Wu, J. Cheng, S. Huang, Y. Ke, Y. Lu, and Y. Xu, "Path problems in temporal graphs," *Proceedings of the VLDB Endowment*, vol. 7, no. 9, pp. 721–732, 2014.
- [81] S. Wang, W. Lin, Y. Yang, X. Xiao, and S. Zhou, "Efficient route planning on public transportation networks: A labelling approach," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 967–982.
- [82] B. Roberts and D. P. Kroese, "Estimating the number of st paths in a graph," *J. Graph Algorithms Appl.*, vol. 11, no. 1, pp. 195–214, 2007.
- [83] L. G. Valiant, "The complexity of enumeration and reliability problems," *SIAM Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979.
- [84] A. Derrow-Pinion, J. She, D. Wong, O. Lange, T. Hester, L. Perez, M. Nunkesser, S. Lee, X. Guo, B. Wiltshire *et al.*, "Eta prediction with graph neural networks in google maps," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 3767–3776.
- [85] H. Yuan, G. Li, Z. Bao, and L. Feng, "Effective travel time estimation: When historical trajectories over road networks matter," in *Proceedings of the 2020 acm sigmod international conference on management of data*, 2020, pp. 2135–2149.
- [86] B. Yang, C. Guo, and C. S. Jensen, "Travel cost inference from sparse, spatio temporally correlated time series using markov models," *Proceedings of the VLDB Endowment*, vol. 6, no. 9, pp. 769–780, 2013.
- [87] H. Kellerer, U. Pferschy, and D. Pisinger, "Multidimensional knapsack problems," in *Knapsack problems*. Springer, 2004, pp. 235–283.
- [88] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—i," *Mathematical programming*, vol. 14, no. 1, pp. 265–294, 1978.